EVENT SYSTEM

OVERVIEW

The event system relies primarily on the scripts GameEventManager.cs and EventType.cs. The event type script holds a list of all the events in the game in a public enum named “E” (for brevity). At the beginning of the game, the game event manager creates a new Unity Event for each element in the enum. It stores each Unity Event with its associated enum in an array of GameEvent objects, which is a custom class. That way, the game event manager can later return the events associated with each value of the enum.

USING THE EVENT SYSTEM

Invoking An Event

In order to invoke an event, you can use the following line, replacing MyEvent with the name of the event you want to invoke:

GameEventManager.current.GetEvent(EventType.E.MyEvent).Invoke();

Listening For An Event

In order to listen for an event, you can add a listener to a class that will call a function whenever the event is called. To do that, add a line like this to your script, replacing "MyEvent" with the name of the event you want to listen for and "SomeFunction" with the name of the function you want to be called:

GameEventManager.current.GetEvent(EventType.E.MyEvent).AddListener(SomeFunction);

Note: Don't include parentheses at the end of the function's name.

HOW TO ADD AN EVENT

1. Add a unique event name at the bottom of the "E" enum in the EventType.cs file.
2. Invoke the event wherever in the game you want the event to occur. (See "Invoking An Event" in the previous section.)

INTERNAL API

This is an important public function associated with the GameEventManager class.

UnityEvent GetEvent(EventType.E eventType)

Returns the event associated with the input “EventName”. See the section “Using the Event System” for more information.

-Graydon Bruyn, May 2026